Lecture 3: HOFs, File I/O, Exceptions, Modules, Testing

Instructor: Jordan Schwartz

Slides adapted from Tony Liu

Logistics

- Check out Ed posts for more info from last lecture
- HW1 is due one week from today!
  - You will need the testing information from today's lecture to complete the HW
  - At 11:20am today, we are jumping to testing! If time we will return to whatever topic we were on, otherwise I will post follow ups on Ed

Additionally, you will be submitting a test file called `HW1_test.py` that you make from scratch. In this file, you will use the unittest library to write unit tests for *5 different functions*.

```
These unittests don't need to be comprehensive, meaning they cover every possible edge case, but they should be more interesting than testing an obvious input and output when possible. Aim for an edge case or to test a specific condition or behavior.
```

Logistics contd.

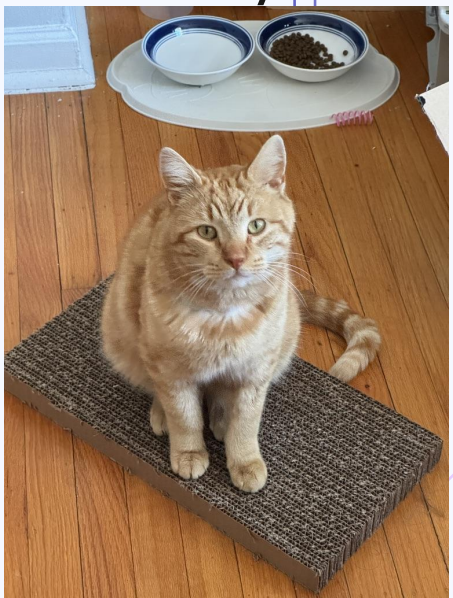
- HW2 will be released sometime next week and requires information from lecture 5 (turtle graphics, pygame, game development)
  - This lecture will be given by our TAs, Anushka and Suhani, because I will be at SIGCSE!

Introductions

Name

If you could have a pet anything, what would you pick?

This is Cheese!



Week 4

- HOFs
- File I/O

- Exceptions
- Modules
- Testing
- ~~Scripting~~ bumped to next week!

Anonymous Functions

Lambda functions

- Are anonymous functions → one line functions defined **without** a name

```
lambda <parameter list>: expression
==
def no_name(<parameter list>):
    return expression
```

[code demo]

Higher Order Functions

What data type are functions?

```
def greeting():
    print("hello")
# without parens, functions are objects
# nothing is telling the function to evaluate
>>> type(greeting)
<class 'function'>
```

🔑: we can treat functions as objects. And what can we do with objects?

[code demo]

[code demo]

Filter, Map, Reduce (a touch of functional programming)

Filter	Map	Reduce

`filter(f, seq)`
is equivalent to:
`[elem for elem in seq if f(elem)]`
	`map(f, seq)`
	is equivalent to:
	`[f(elem) for elem in seq]`
		is a function that takes in an aggregation function and a sequence and repeatedly accumulates elements from the sequence using the aggregation function.
		`from functools import reduce`
		`reduce(f, seq)`
		is roughly equivalent to:
		`result = seq[0]`
		`for elem in seq[1:]:`
		`  result = f(result, elem)`

Check In

M1: `list(filter(lambda x: x % 2 == 0, [0, 1, 2, 3]))`
M2: `list(map(lambda y: y % 2 == 0, [0, 1, 2, 3]))`
M3: `reduce(lambda x, y: x + y, [[0], [1], [2], [3]])`

Match to -
A: `[True, False, True, False]`
B: `[0, 2]`
C: `[0, 1, 2, 3]`
D: Error

File I/O and context management

Basic I/O

- input(): read from terminal stdin, stop after newline
- print(str): display the str in terminal, stdout

File Objects

- File objects expose an API to an underlying file resource

- open() returns a file object, with 2 arguments: open(filename, mode)
- close(): destructor for file object

File modes

Character	Meaning
'r'	open for reading (default)
'w'	open for writing, overwriting if file already exists
'x'	open for exclusive creation, fail if exists
'a'	open for writing, appending to the end of the file if it exists
'b'	open in binary mode
't'	open in text mode (default)
'+'	open for updating (reading and writing)

File reading and writing

[code demo]

Context management: with keyword

- with keyword allows for resource management in enclosed context
  - For file I/O, easy management/cleanup of file objects
- tedious since it is a common code pattern, can be error prone

```
try:
    # ...perform file operations...
    f = open("python_zen.txt")
    for line in f:
        print(line.strip())
except:
    # ...handle exceptions...
    pass
finally:
    # don't forget to clean up file
object!
    f.close()
```

with Simplifies Things

```
# same as above, with context
management
with open("python_zen.txt") as f:
    for line in f:
        print(line.strip())
        #"line\n"
    #raise Exception
#print(f.closed)
```

Context Management

- a context manager defines two magic functions:
  - __enter__: called upon entering the with context
  - __exit__: called upon exiting the with context, even if an exception is thrown
- allows for boilerplate definition of clean-up actions and resource management
  - thread creation/deletion, database connection open/close...

Hints: `.read()`, `.split()`

Check In

Often, when someone is trying to hack into a file or system, they will place what we call (evil) “shellcode” into a script to be run in place of what was supposed to be run.

C14: Given the file, such as, `imsafepleaseopenme.txt`, can you write a function to check if it contains the word “`shellcode`”? If “`shellcode`” is in the file it should return True, otherwise False.

Exceptions

Exceptions

- Errors detected during program execution
  - Different from syntax errors, which are detected by the interpreter before runtime
- Examples

```
>>> next(it)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
```

```
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

```
>>> 1/0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
```

Handling Exceptions

- We handle exceptions with `try except` blocks
  - Just like Java's try-catch blocks

- if no exceptions, except blocks skipped and execution continues
- if exception occurs, rest of try block is skipped
- If exception type match is found, that block is executed and execution continues
- If no match for the Exception is found, execution is stopped with error message

Recap: input() (we saw in File I/O)

```
>>> help(input)
Help on built-in function input in module builtins:

input(prompt='', /)
    Read a string from standard input.  The trailing newline is stripped.

    The prompt string, if given, is printed to standard output without a
    trailing newline before reading input.

    If the user hits EOF (*nix: Ctrl-D, Windows: Ctrl-Z+Return), raise EOFError.
    On *nix systems, readline is used if available.
```

Try Except Example

```
try:
    exp = int(input("Input a number: "))
    print(2. ** exp)

except OverflowError:
    print('Can\'t compute that number due to overflow')
except ValueError:
    print('Input is not a number')
except:
    print('Catches all other exceptions, careful using this!')
```

[Code demo]

Exception Clean Up

- *else* in a try-except is entered if the try clause does not raise an exception
- *finally* will execute before the entire try-except statement concludes, whether or not an exception is raised

[Code demo]

Raising Exceptions

The *raise* keyword is used to force a specific exception to occur

[Code demo]

raise can also be used to re-raise an Exception within a try-except block

[Code demo]

Creating our own exceptions

can be easily done by subclassing the base Exception class

[Code demo]

Modules

Scripting as we know it

- .py files
- Executed by running `python3 script.py`

Modules

- When we start a REPL in the terminal and close it, the code we wrote is lost (not available the next time we open a REPL)
- Writing a `.py` file creates a module - this saves our function and class definitions
- Can be imported into other modules and can import other modules
- Modules are saved by their file name minus the `.py` and can be accessed by `__name__`

Modules Contd.

- May contain executable statements along with definitions
  - These are run upon importing the module
- Modules define a namespace
  - Global variables live within the module
  - Access them from outside the module via `module_name.attr`
- Libraries are a collection of modules

Importing Modules

- By convention, place imports at the top

```
>>> from fibo import fib, fib2
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

```
>>> import fibo as fib
>>> fib.fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

```
>>> from fibo import *
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

```
>>> from fibo import fib as fibonacci
>>> fibonacci(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

```
>>> import fibo
>>> fibo.fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> fibo.fib2(100)
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
>>> fibo.__name__
'fibo'
```

Standard Modules

https://docs.python.org/3/library/

Some popular ones:

- `import random` (pretty common, if time we will come back and play around with it)
- `import functools` (we saw for HOFs)
- `import pickle` (we will see later maybe)

Scripting in more detail

- Script: designed to run directly and perform a specific action from the command line
- Module: designed to be imported and reused within other Python programs
- argparse is a standard library that provides high-level argument parsing
  - contrast with sys module, which handles low level details like script flags

Check in

Discuss with your group:
What other modules do you know about, or think might exist, or, even, think would be useful when coding in Python?

Write at least one down in L11

Executing modules as scripts

Running a python module with

```
>>> python3 fibo.py <arguments>
```

Runs the code as if it was imported with `__name__` set to `"__main__"`

Adding

```
if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))
```

Makes the file usable as a script in addition to being an importable module because the code that parses the command line only runs if the module is executed as the “main” file

Outputs for different calls to the file

```
if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))
```

```
$ python fibo.py 50
0 1 1 2 3 5 8 13 21 34
```

```
>>> import fibo
>>>
```

Testing

Testing

- Naively/Intuitively: write `print` statements to verify output, etc.
  - These get deleted when code is finalized
- Instead we can use `assert`
  - Statements to debug beyond print statements
  - Syntax: `assert bool_expression, description`

Unit Tests

- Unit tests check the functionality of specific pieces of code for correct behavior, in isolation
- Contrast with integrations tests:
  - Unit test: isolates and checks a single piece of code (eg. a function)
  - Integration tests: test how parts of the application work together as a whole
- Use the `unittest` library!

Testing using `unittest`

- A standard library for unit testing in an object-oriented manner

- Subclass the `unittest.TestCase` class to write custom test cases
- Any method defined in the class starting with `test_` is considered a test case
- Numerous `assert...` statements, see doc for full details
  - Most common is `assertEqual(a, b)`: checks that `a == b`

[code demo]

Check in

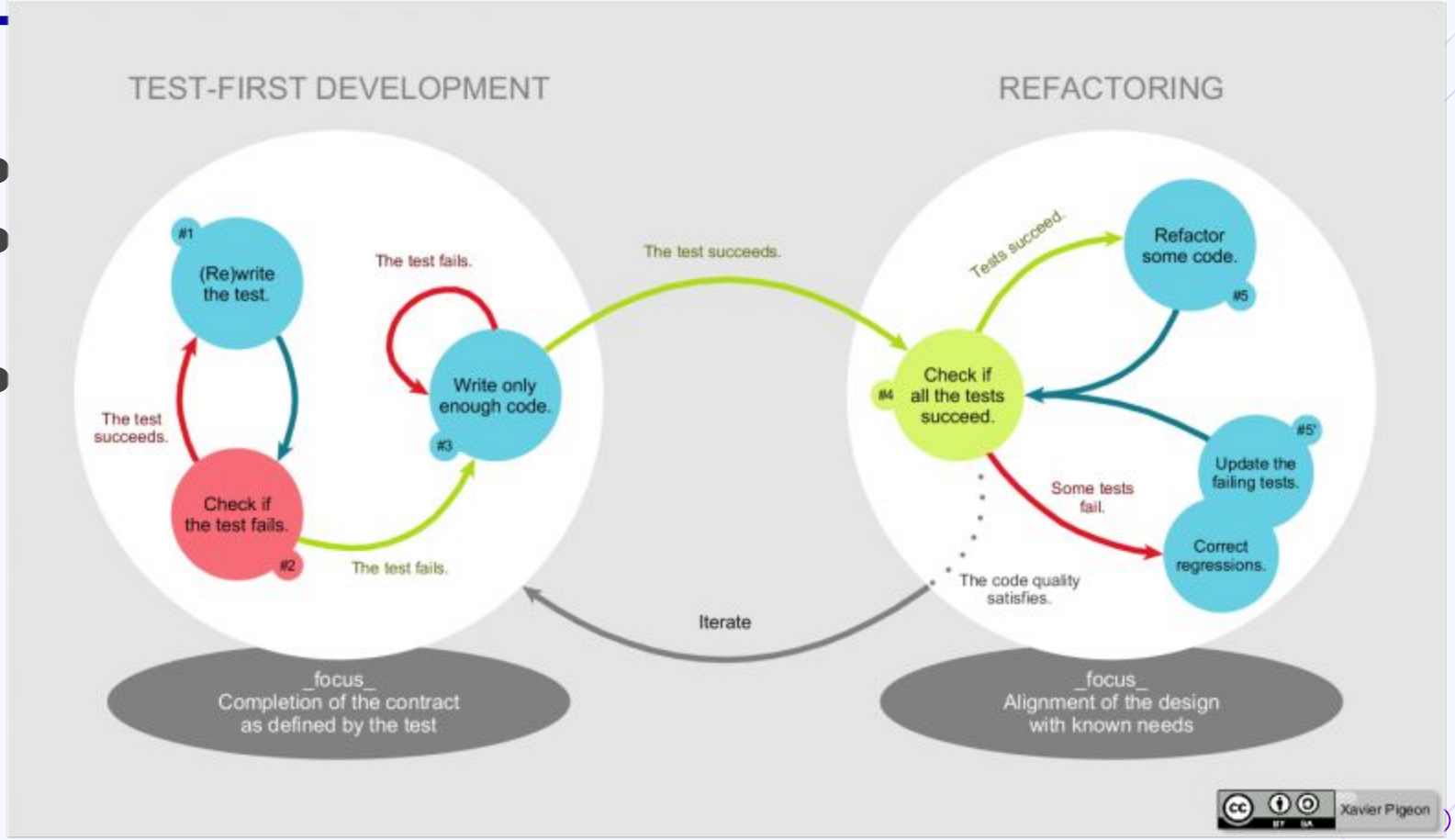
See code in lecture4.py file.

In C12: Write up to 3 unittests that would catch the main bug in this function.

Hint: use the assertTrue and assertFalse functions

Bonus points: write the entire testing file, including the imports and class header

T


TEST-FIRST DEVELOPMENT
REFACTORING
#1
(Re)write the test.
The test fails.
The test succeeds.
Write only enough code.
#3
The test succeeds.
Check if the test fails.
#2
The test fails.
Tests succeed.
Refactor some code.
#5
#4
Check if all the tests succeed.
#5'
Update the failing tests.
Some tests fail.
Correct regressions.
The code quality satisfies.
Iterate
focus
Completion of the contract as defined by the test
focus
Alignment of the design with known needs
Xavier Pigeon

Thanks!

Do you have any questions?

youremail@freepik.com
+34 654 321 432
yourwebsite.com



Instructions for use

If you have a free account, in order to use this template, you must credit **Slidesgo** by keeping the **Thanks** slide. Please refer to the next slide to read the instructions for premium users.

As a Free user, you are allowed to:

- Modify this template.
- Use it for both personal and commercial projects.

You are not allowed to:

- Sublicense, sell or rent any of Slidesgo Content (or a modified version of Slidesgo Content).
- Distribute Slidesgo Content unless it has been expressly authorized by Slidesgo.
- Include Slidesgo Content in an online or offline database or file.
- Offer Slidesgo templates (or modified versions of Slidesgo templates) for download.
- Acquire the copyright of Slidesgo Content.

For more information about editing slides, please read our FAQs or visit our blog:
https://slidesgo.com/faqs and **https://slidesgo.com/slidesgo-school**